C.KRITHIKA - 9143

EXP NO: 08	TEST THE SOFTWARE SYSTEM FOR ALL THE SCENARIOS IDENTIFIED AS PER THE USE CASE DIAGRAM

PROJECT TITLE: **STUDENT ATTENDANCE MANAGEMENT SYSTEM**

1. TESTING APPROACH

Test Type	Purpose
Unit Testing	To verify individual methods and classes work correctly
Integration Testing	To ensure that different components interact properly
System Testing	To validate the complete system functionality

2. TEST CASES FOR USE CASES

Use Case 1: Mark Attendance

Test Case 1.1: Successfully Mark Attendance

- **Test ID:** TC_01
- **Description:** Verify that attendance can be successfully recorded
- **Precondition:** Database is running and GUI is open
- **Test Steps:**
    1. Enter student name
    2. Enter subject
    3. Enter date
    4. Select status (Present/Absent)
    5. Click Submit
- **Expected Result:** Attendance data should be stored in the database
- **Actual Result:** Attendance inserted successfully (**Pass**)

Test Case 1.2: Missing Input Fields

- **Test ID:** TC_02
- **Description:** Check system behavior when input fields are empty
- **Precondition:** GUI is open
- **Test Steps:**
    1. Leave student name empty
    2. Click Submit
- **Expected Result:** System should prevent insertion or show an error message
- **Actual Result:** Invalid input handled properly (**Pass**)

Use Case 2: Store Attendance in Database

Test Case 2.1: Verify Data Storage

- **Test ID:** TC_03
- **Description:** Verify that entered data is stored in the database
- **Precondition:** Attendance data is submitted
- **Test Steps:**
    1. Open SQL Server
    2. Execute query:

SELECT * FROM attendance;

- **Expected Result:** Inserted record should be displayed
- **Actual Result:** Data displayed successfully (**Pass**)

Use Case 3: GUI Interaction

Test Case 3.1: Status Selection

- **Test ID:** TC_04
- **Description:** Verify radio button selection for status
- **Precondition:** GUI is open
- **Test Steps:**
    1. Select Present
    2. Submit data

- **Expected Result:** Status should be stored as Present
- **Actual Result:** Correct status stored (**Pass**)

Use Case 4: View Attendance Percentage

Test Case 4.1: Calculate Percentage and Eligibility

- **Test ID:** TC_05
- **Description:** Verify attendance percentage and eligibility calculation
- **Precondition:** Attendance data available in database
- **Test Steps:**
    1. Open SQL Server
    2. Execute query:

```
SELECT
   name,
   COUNT(*) AS total_classes,
   SUM(CASE WHEN status = 'Present' THEN 1 ELSE 0 END) AS present_days,
   (SUM(CASE WHEN status = 'Present' THEN 1 ELSE 0 END) * 100.0 / COUNT(*)) AS
percentage,
   CASE
      WHEN (SUM(CASE WHEN status = 'Present' THEN 1 ELSE 0 END) * 100.0 /
COUNT(*)) >= 75
      THEN 'Eligible for Exam'
      ELSE 'Not Eligible for Exam'
   END AS eligibility
FROM attendance
GROUP BY name;
```

- **Expected Result:** Correct percentage and eligibility should be displayed for all students
- **Actual Result:** Correct output displayed (**Pass**)

3. UNIT TESTING

Unit testing is performed on individual methods.

- The add() method in the Attendance class is tested
- The method successfully inserts attendance data into the database

Expected Result: Method should execute without errors
Actual Result: Passed

4. INTEGRATION TESTING

Integration testing ensures that the Java application and database work together.

- DBConnection and Attendance classes are tested together
- Data is successfully inserted into SQL Server database

Expected Result: Database connection established and data stored correctly
Actual Result: Passed

5. SYSTEM TESTING

System testing is performed to validate the entire system.

- GUI interface is tested
- Data entry and submission is verified
- Database storage is confirmed
- Attendance percentage calculation is verified

Expected Result: System should work according to requirements
Actual Result: System working correctly (**Passed**)

6. TEST SUMMARY

Test Type	Status
Unit Testing	Passed
Integration Testing	Passed
System Testing	Passed

RESULT:

Thus, the Student Attendance Management System was successfully tested for all the identified scenarios. The system correctly performs data entry, storage, and attendance evaluation as per the use case diagram.

C.KRITHIKA - 9143

EXP NO: 01	IDENTIFIES A SOFTWARE SYSTEM THAT NEEDS TO BE DEVELOPED

PROJECT TITLE : STUDENT ATTENDANCE MANAGEMENT SYSTEM

OBJECTIVE

The Objective of this project is to design and develop a Student Attendance Management System that helps in maintaining attendance records efficiently. The system aims to reduce manual work, improve accuracy, and provide easy access to attendance information. It also helps in managing student data in a well-organized and systematic manner.

PROBLEM STATEMENT

In many educational institutions, attendance management is still a time-consuming process even when digital systems are used. Maintaining accurate records, generating reports, and handling large amounts of student data becomes difficult without a proper structured system. Errors may occur while updating attendance, and retrieving previous records is not always easy. Hence, there is a need for a well-organized and efficient Student Attendance Management System that can simplify attendance handling, reduce errors, and improve overall efficiency.

SCOPE OF THE PROJECT

- The system allows storing and managing student details.
- It helps in marking and maintaining attendance digitally.
- The system supports easy retrieval of attendance records.
- It reduces manual work and paperwork.
- It provides accurate attendance calculation.
- It helps in generating attendance reports easily.
- The system is simple, user-friendly, and can be enhanced in future.

RESULT:

The Student Attendance Management System was successfully developed and implemented. The system makes attendance management simple, accurate, and time-saving. It helps in reducing manual effort and provides an efficient way to store and retrieve student attendance details.

EXP NO: 02	DOCUMENT THE SOFTWARE REQUIREMENTS SPECIFICATION (SRS) FOR THE IDENTIFIED SYSTEM

1. INTRODUCTION

Software Requirements Specification (SRS) is a document that clearly defines the requirements of a software system. It is prepared using IEEE 830 standards to describe functional and non-functional requirements. In this experiment, the SRS is developed for the Student Attendance Management System to ensure efficient and error-free attendance management.

PROJECT TITLE: STUDENT ATTENDANCE MANAGEMENT SYSTEM

1.1 PROCESS MODEL USED – WATERFALL MODEL

DESCRIPTION

The Waterfall Model is a sequential software development process in which each phase is completed before moving to the next phase. It follows a **step-by-step approach** such as requirement analysis, system design, implementation, testing, and maintenance. This model is suitable for projects where **requirements are clearly defined and stable.**

In the Student Attendance Management System, the Waterfall model is used because the system requirements are well understood and do not change frequently. This model helps in systematic development and easy documentation.

1.2 PURPOSE

The Student Attendance Management System is designed to automate the process of maintaining student attendance in educational institutions. The system helps in recording, managing, and retrieving attendance details efficiently, thereby **reducing manual work and human errors.**

1.3 SCOPE

- Allows staff to mark student attendance
- Stores attendance records securely
- Generates attendance reports
- Reduces paperwork
- Provides easy access to attendance data

1.4 DEFINITIONS, ACRONYMS, AND ABBREVIATIONS

- **SAMS** – Student Attendance Management System
- **Admin** – System administrator
- **Staff** – Person who marks attendance
- **Student** – Person whose attendance is recorded

2. OVERALL DESCRIPTION

2.1 PRODUCT PERSPECTIVE

- Standalone application
- Developed using Java
- Used for managing student attendance
- Replaces manual attendance system

2.2 PRODUCT FUNCTIONS

- User login and authentication
- Student record management
- Attendance marking
- Attendance report generation
- Data storage and retrieval

2.3 USER CHARACTERISTICS

- **Admin** – Manages system and users
- **Staff** – Marks attendance
- **Student** – Views attendance

2.4 CONSTRAINTS

- Only authorized users can access the system
- System availability is required for proper functioning.
- Attendance can be marked only once per session
- Database connection is mandatory

3. SPECIFIC REQUIREMENTS

3.1 FUNCTIONAL REQUIREMENTS

3.1.1 User Authentication

- System shall allow user login and validation.

3.1.2 Student Management

- System shall allow adding, updating, and deleting student details.

3.1.3 Attendance Management

- System shall allow marking attendance.
- System shall store attendance and generate reports.

3.2 NON-FUNCTIONAL REQUIREMENTS

- System shall be user-friendly
- System shall provide fast response
- System shall support multiple users
- System shall ensure data security
- System shall be reliable and scalable

3.3 INTERFACE REQUIREMENTS

- **User Interface:** Java-based Graphical User Interface (GUI) using Swing
- **Database:** SQL Server
- **Software:** Java, JDBC (SQL Server Driver), SQL Server Management Studio (SSMS)
- **Hardware:** Minimum 4GB RAM

RESULT:

Thus, the Software Requirements Specification (SRS) for the Student Attendance Management System was successfully documented by following IEEE 830 standards. The document clearly defines the functional and non-functional requirements, system constraints, and assumptions required for effective system development.

EXP NO: 03	IDENTIFY USE CASES AND DEVELOP THE USE CASE MODEL

<u>PROJECT TITLE:</u> STUDENT ATTENDANCE MANAGEMENT SYSTEM

USE CASE DESCRIPTION:

The Student Attendance Management System is designed to simplify and automate the process of attendance management in educational institutions. The system allows different types of users to interact with it based on their roles and responsibilities. These users are represented as actors in the Use Case diagram.

The system mainly consists of three actors:

- **Admin** – Admin manages the entire system and controls user activities
- **Staff** – Staff is responsible for marking and updating attendance
- **Student** – Student can view attendance details and percentage

Each actor interacts with the system through specific use cases, which represent the functionalities provided by the system.

USE CASES:

Admin:

• Manage Users (Add, Update, Delete student and staff details)

• Generate Attendance Reports

• Monitor system activities

Staff:

• Mark Attendance

• Edit or Update Attendance

• View Attendance Records

Student:

• View Attendance

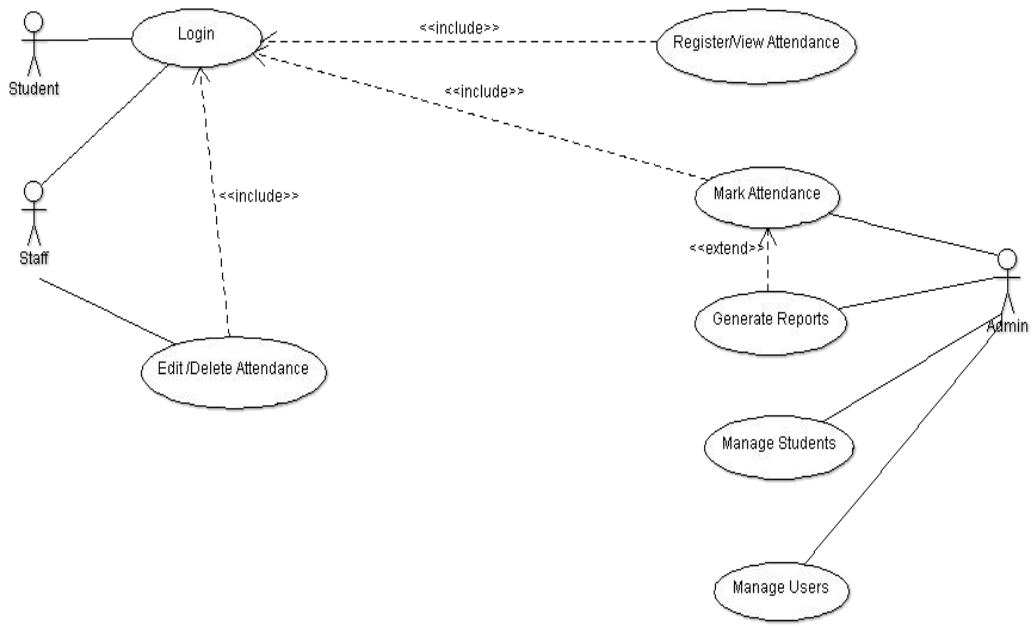
• Check Attendance Percentage

• Verify eligibility status

System Interaction:

- Admin controls and manages overall system operations
- Staff interacts with the system to store attendance data
- Student interacts with the system to retrieve attendance data
- System processes requests and communicates with the database

USE CASE DIAGRAM FOR STUDENT ATTENDANCE MANAGEMENT SYSTEM



RESULT:

Thus, the actors (Admin, Staff, Student) were successfully identified and the Use Case model was developed to represent system interactions.

C.KRITHIKA - 9143

EXP NO: 04	IDENTIFY THE CONCEPTUAL CLASSES AND DEVELOP A DOMAIN MODEL AND ALSO DERIVE A CLASS DIAGRAM FROM THAT

PROJECT TITLE: **STUDENT ATTENDANCE MANAGEMENT SYSTEM**

DESCRIPTION:

In the Student Attendance Management System, conceptual classes represent the real-world entities involved in the system.

The following conceptual classes are identified:

• **Student** – Represents students whose attendance is recorded
• **Staff** – Represents faculty members who mark attendance
• **Admin** – Represents system administrator
• **Attendance** – Stores attendance details such as name, subject, date, and status
• **Course** – Represents subjects or courses

ATTRIBUTES AND METHODS:

Student

• Attributes: studentId, name, email, department
• Methods: login(), viewAttendance()

Staff

• Attributes: staffId, name, department, email
• Methods: markAttendance(), editAttendance()

Admin

• Attributes: adminId, name, email
• Methods: manageUsers(), generateReport()

Attendance

• Attributes: name, subject, date, status
• Methods: add()

Course

• Attributes: courseId, courseName, courseCode
• Methods: addStudent(), assignStaff()

Domain Model:

• Student is associated with multiple attendance records

• Staff manages attendance data

• Course contains students and staff

• Admin controls the entire system

Relationships:

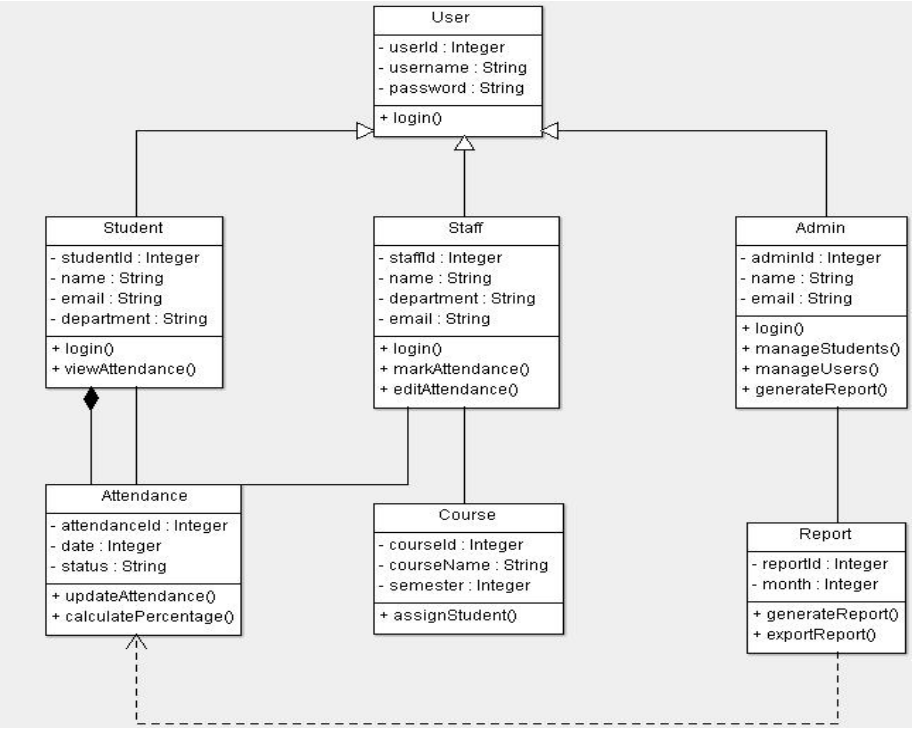
• Association between Student and Attendance

• Association between Staff and Attendance

• Aggregation between Course and Student

• Inheritance from User to Admin, Staff, and Student

CLASS DIAGRAM FOR STUDENT ATTENDANCE MANAGEMENT SYSTEM



RESULT:

Thus, the conceptual classes were identified and the domain model and class diagram were successfully developed showing system structure and relationships.

EXP NO: 05	USING THE IDENTIFIED SCENARIOS, FIND THE INTERACTION BETWEEN OBJECTS AND REPRESENT THEM USING UML SEQUENCE AND COLLABORATION DIAGRAMS

PROJECT TITLE: **STUDENT ATTENDANCE MANAGEMENT SYSTEM**

DESCRIPTION:

In the Student Attendance Management System, different objects interact with each other to perform operations like marking attendance and viewing attendance records. These interactions are important to understand how data flows within the system. Sequence diagrams and Collaboration diagrams are used to represent these interactions clearly.

1. SEQUENCE DIAGRAM

Sequence diagrams show how objects communicate with each other in a particular sequence over time.

A. Mark Attendance

Actors: Staff, GUI, Attendance Class, Database

STEPS:

• Staff enters student name, subject, date, and selects status

• GUI collects the input data from the user

• GUI sends the data to the Attendance class

• Attendance class processes the input

• Attendance class establishes connection with database

• Data is inserted into the attendance table

• Database confirms successful storage

• GUI displays confirmation message to the staff

B. View Attendance

Actors: Student, System, Database

STEPS:

• Student requests to view attendance

• System receives the request

• System sends query to database

• Database processes the query

• Attendance data is retrieved

• System displays attendance details to the student

2. COLLABORATION DIAGRAM

Collaboration diagrams represent how objects interact with each other to complete a task.

A. Mark Attendance

• Staff interacts with GUI interface

• GUI forwards data to Attendance class

• Attendance class communicates with DBConnection

• DBConnection connects to database

• Attendance data is stored in database

B. View Attendance

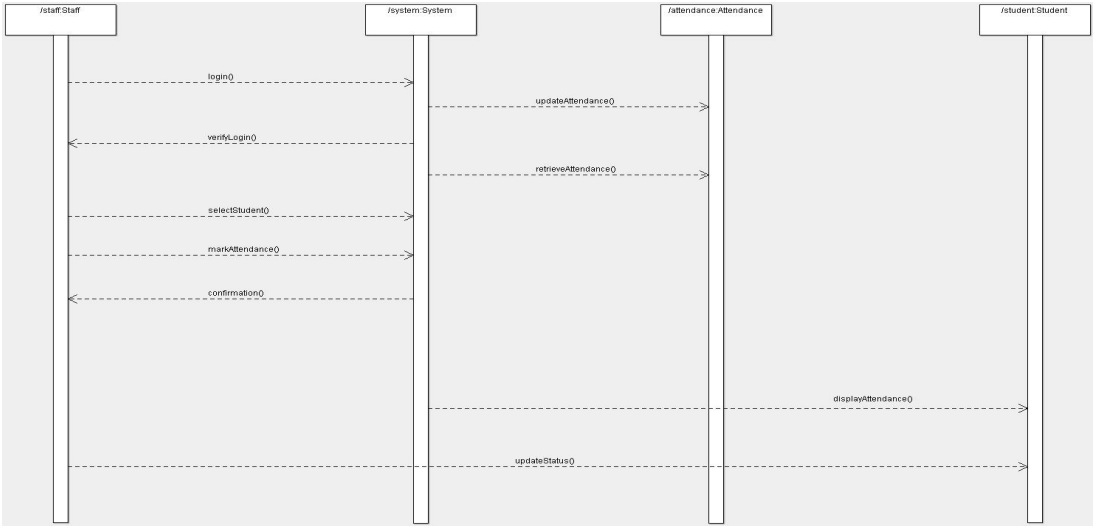
• Student sends request to system

• System interacts with database

• Database returns required data

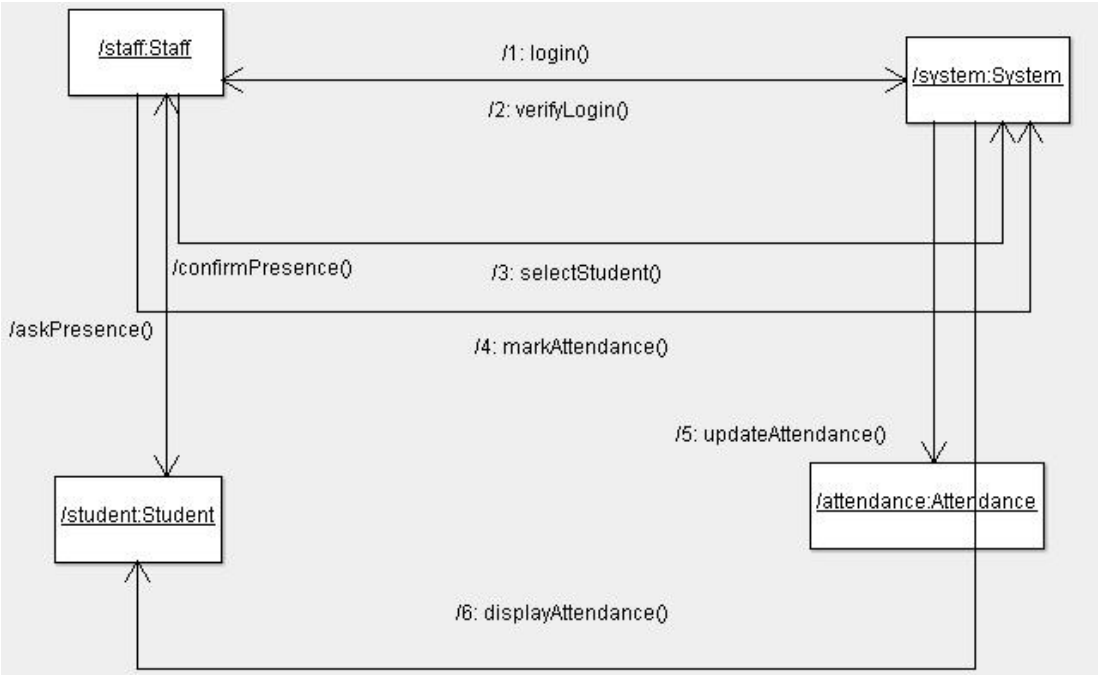
• System displays output to student

SEQUENCE DIAGRAM FOR STUDENT ATTENDANCE MANAGEMENT SYSTEM



C.KRITHIKA - 9143

COLLABORATION DIAGRAM FOR STUDENT ATTENDANCE MANAGEMENT SYSTEM



RESULT:

Thus, the interactions between objects were successfully identified and represented using Sequence and Collaboration diagrams.

EXP NO: 06	DRAW RELEVANT STATE CHART AND ACTIVITY DIAGRAMS FOR THE SAME SYSTEM

PROJECT TITLE: **STUDENT ATTENDANCE MANAGEMENT SYSTEM**

DESCRIPTION:

In the Student Attendance Management System, different UML diagrams are used to represent system behavior, process flow, and physical architecture. State Chart and Activity diagrams describe the dynamic behavior of the system, while the Deployment diagram represents how the system is physically deployed.

1. STATE CHART DIAGRAM (Attendance Status)

The State Chart Diagram shows the different states of attendance and how it changes based on user actions.

States:

• Not Marked – Initial state before attendance is entered
• Present – Student is marked present
• Absent – Student is marked absent

Transitions:

• Staff selects student → moves from Not Marked state
• Staff marks attendance → changes to Present or Absent
• Data is saved → state remains stored in system

2. ACTIVITY DIAGRAM

Activity diagrams represent the flow of activities in the system.

A. Mark Attendance

STEPS:

• Staff logs into the system
• Enters student details
• Selects subject and date
• Chooses attendance status
• Clicks submit button

• System processes the data

• Data is stored in database

• Confirmation message is displayed

B. View Attendance

STEPS:

• Student logs into the system

• Requests attendance details

• System sends request to database

• Database retrieves records

• System displays attendance details

3. DEPLOYMENT DIAGRAM

The Deployment Diagram shows the physical structure of the system, including hardware and software components.

Nodes Involved:

• **Client System (User Machine**)

- Runs Java Swing GUI
- Used by Admin, Staff, Student

• **Application Layer (Java Program)**

- Contains business logic
- Processes user inputs
- Connects to database using JDBC

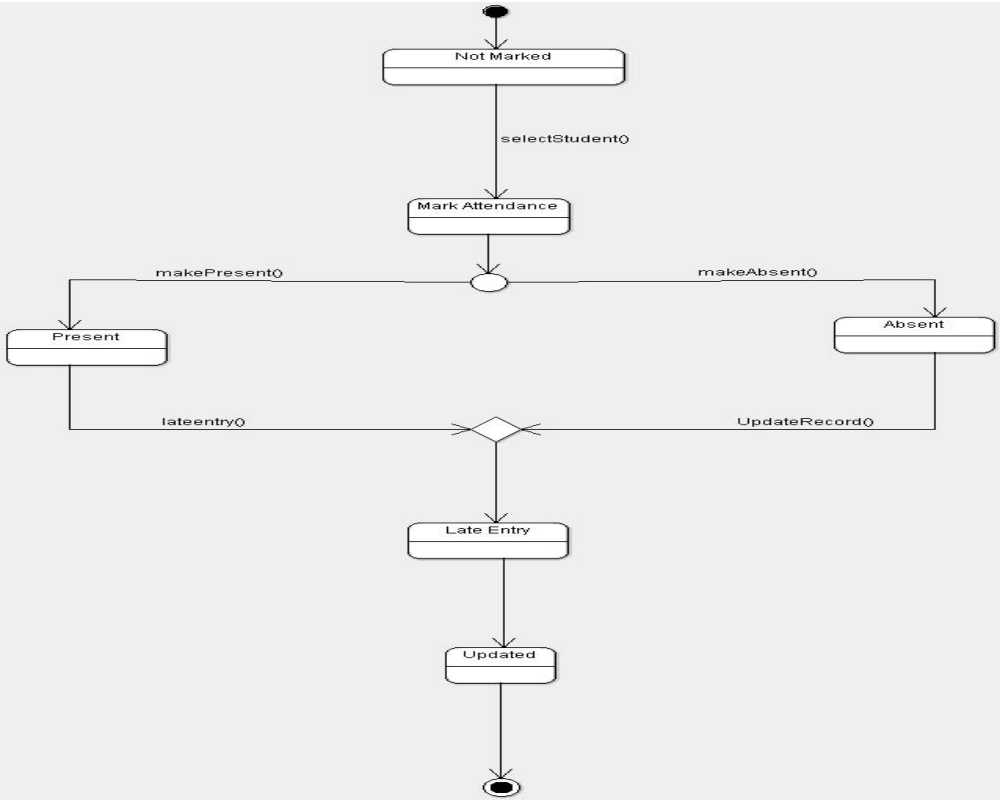
• **Database Server (SQL Server)**

- Stores attendance data
- Executes SQL queries

Connections:

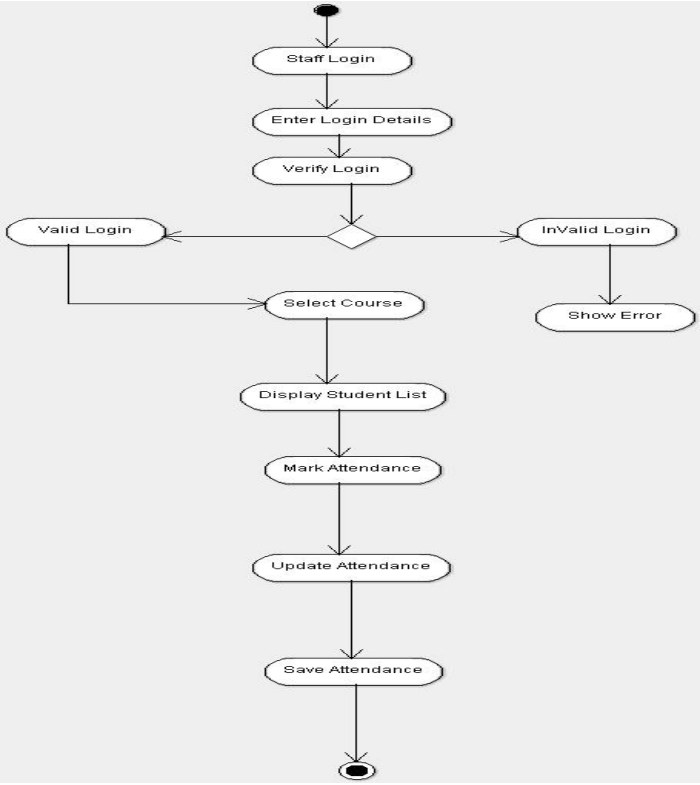
- Client system interacts with Java application
- Java application communicates with database using JDBC
- Database server stores and retrieves attendance data

STATE CHART DIAGRAM FOR STUDENT ATTENDANCE MANAGEMENT SYSTEM

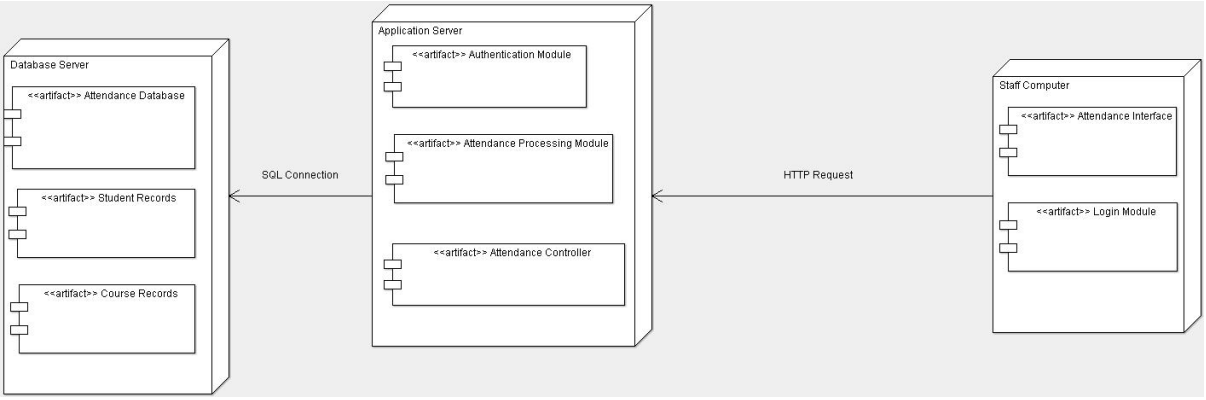


C.KRITHIKA - 9143

ACTIVITY DIAGRAM FOR STUDENT ATTENDANCE MANAGEMENT SYSTEM



DEPLOYMENT DIAGRAM FOR STUDENT ATTENDANCE MANAGEMENT SYSTEM



RESULT:

Thus, the system behavior, workflow, and deployment architecture were successfully represented using State Chart, Activity, and Deployment diagrams.

EXP NO: 07	IMPLEMENT THE SYSTEM AS PER THE DETAILED DESIGN

PROJECT TITLE: **STUDENT ATTENDANCE MANAGEMENT SYSTEM**

1. TECHNOLOGY STACK

- User Interface: Java Swing (GUI)
- Programming Language: Java
- Database: SQL Server
- JDBC: SQL Server Driver
- Software: Java, SQL Server Management Studio (SSMS)
- Hardware: Minimum 4GB RAM

2. IMPLEMENTATION STEPS

A. Setup SQL Server Database

1. **Open SQL Server Management Studio (SSMS)**
2. **Create Database and Table:**

```
CREATE DATABASE attendance_db;
GO

USE attendance_db;
GO

DROP TABLE attendance;

CREATE TABLE attendance (
   name VARCHAR(50),
   subject VARCHAR(50),
   date VARCHAR(20),
   status VARCHAR(10)
);

SELECT * FROM attendance;
```

3. **Attendance Report Query:**

```
SELECT
   name,
   COUNT(*) AS total_classes,
   SUM(CASE WHEN status = 'Present' THEN 1 ELSE 0 END) AS present_days,
   (SUM(CASE WHEN status = 'Present' THEN 1 ELSE 0 END) * 100.0 / COUNT(*)) AS
percentage,
   CASE
      WHEN (SUM(CASE WHEN status = 'Present' THEN 1 ELSE 0 END) * 100.0 /
```

```
COUNT(*)) >= 75
      THEN 'Eligible for Exam'
      ELSE 'Not Eligible for Exam'
   END AS eligibility
FROM attendance
GROUP BY name;
```

4. **Insert Sample Data:**

```
INSERT INTO attendance VALUES ('Krithi','OOSE','22-03-2026','Present');
INSERT INTO attendance VALUES ('Arun','OOSE','23-03-2026','Absent');
```

B. Establish Database Connection Using JDBC

This class is used to connect Java application with SQL Server database.

```
import java.sql.*;

public class DBConnection {
   public static Connection getConnection() {
      try {
         Class.forName("com.microsoft.sqlserver.jdbc.SQLServerDriver");

         Connection con = DriverManager.getConnection(

"jdbc:sqlserver://localhost;databaseName=attendance_db;encrypt=true;trustServerCertificate
=true;integratedSecurity=true"
         );

         System.out.println("Connected");
         return con;

      } catch (Exception e) {
         System.out.println("ERROR: " + e);
         return null;
      }
   }
}
```

C. Implement the Core Classes

1. User Base Class

```
public class User {

   public void login() {
      System.out.println("User logged in");
   }
}
```

2. Admin Class

This class handles admin functionalities like managing students and generating reports.

```
import java.util.Vector;

public class Admin extends User {

    private int adminId;
    public String name;
    public String email;

    public Vector<Report> reports;

    public Admin() {
        reports = new Vector<>();
    }

    public void login() {
        System.out.println("Admin logged in");
    }

    public void manageStudents() {
        System.out.println("Managing students");
    }

    public void manageUsers() {
        System.out.println("Managing users");
    }

    public void generateReport() {
        System.out.println("Report generated");
    }
}
```

3. Student Class

Represents student and allows viewing attendance.

```
import java.util.Vector;

public class Student extends User {

    private int studentId;
    private String name;
    public String email;
    public String department;

    public Vector<Attendance> attendanceList;
    public Vector<Course> courses;

    public Student() {
```

```
        attendanceList = new Vector<>();
        courses = new Vector<>();
    }

    public void login() {
        System.out.println("Student logged in");
    }

    public void viewAttendance() {
        System.out.println("Viewing attendance");
    }
}
```

4.Staff Class

Handles attendance marking and editing.

```
import java.util.Vector;

public class Staff extends User {

    private int staffId;
    public String name;
    public String department;
    public String email;

    public Vector<Attendance> myAttendance;
    public Vector<Course> courses;

    public Staff() {
        myAttendance = new Vector<>();
        courses = new Vector<>();
    }

    public void login() {
        System.out.println("Staff logged in");
    }

    public void markAttendance() {
        System.out.println("Attendance marked");
    }

    public void editAttendance() {
        System.out.println("Attendance edited");
    }
}
```

5. Course Class

Manages students and staff assigned to a course.

```
import java.util.Vector;

public class Course {

    private int courseId;
    private String courseName;
    private String courseCode;

    public Vector<Student> students;
    public Vector<Staff> staffList;

    public Course() {
        students = new Vector<>();
        staffList = new Vector<>();
    }

    public void addStudent(Student s) {
        students.add(s);
        System.out.println("Student added to course");
    }

    public void assignStaff(Staff s) {
        staffList.add(s);
        System.out.println("Staff assigned to course");
    }

    public void showCourseDetails() {
        System.out.println("Course details displayed");
    }
}
```

6. Attendance Class

Handles insertion of attendance into database.

```
import java.sql.*;

public class Attendance {

    public void add(String name, String subject, String date, String status) {

        try {
            Connection con = DBConnection.getConnection();

            String q = "INSERT INTO attendance VALUES (?,?,?,?)";
            PreparedStatement ps = con.prepareStatement(q);

            ps.setString(1, name);
            ps.setString(2, subject);
            ps.setString(3, date);
            ps.setString(4, status);
```

```
            ps.executeUpdate();

        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

D. GUI Implementation

This GUI allows user to enter attendance details.

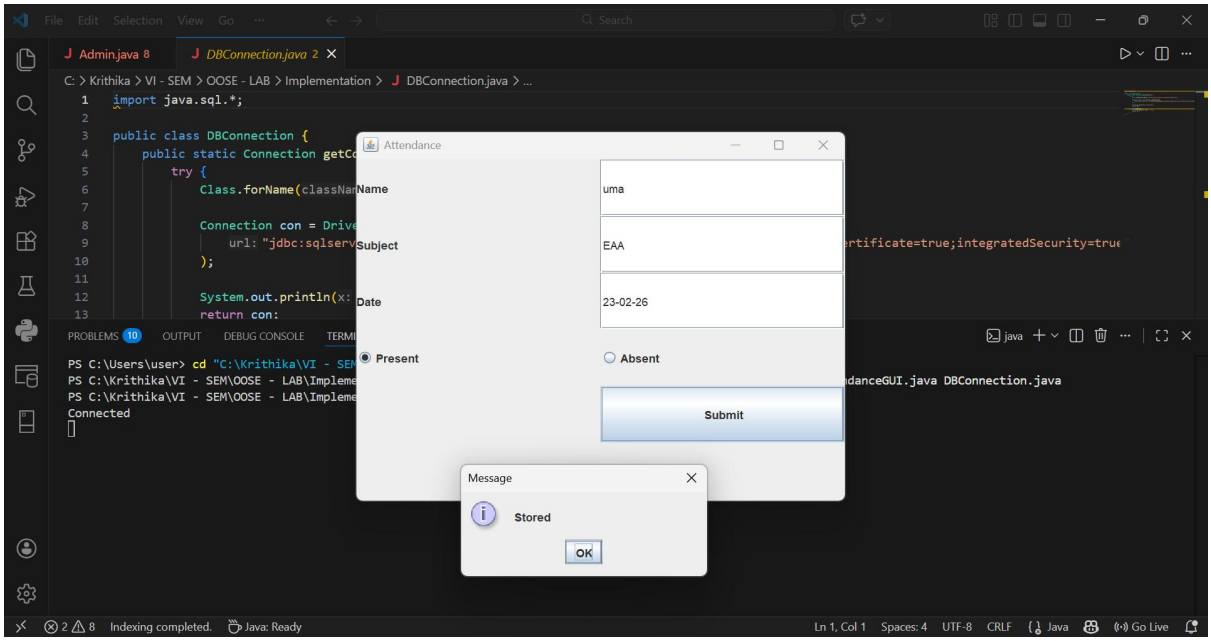
```
import java.awt.*;
import javax.swing.*;

public class AttendanceGUI {

    public static void main(String[] args) {

        JFrame f = new JFrame("Attendance");
        f.setSize(300, 300);
        f.setLayout(new GridLayout(6,2));
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        JTextField name = new JTextField();
        JTextField subject = new JTextField();
        JTextField date = new JTextField();

        JRadioButton p = new JRadioButton("Present");
        JRadioButton a = new JRadioButton("Absent");

        ButtonGroup bg = new ButtonGroup();
        bg.add(p);
        bg.add(a);

        JButton submit = new JButton("Submit");

        f.add(new JLabel("Name"));
        f.add(name);

        f.add(new JLabel("Subject"));
        f.add(subject);

        f.add(new JLabel("Date"));
        f.add(date);

        f.add(p);
        f.add(a);

        f.add(new JLabel(""));
```

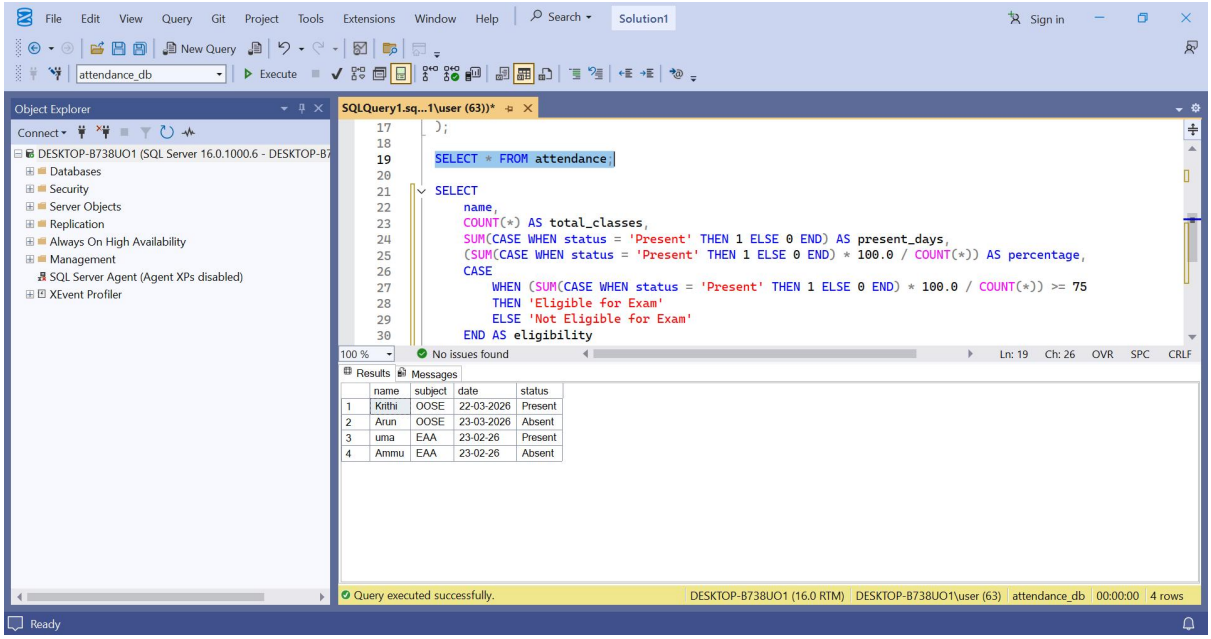
```
        f.add(submit);

        Attendance att = new Attendance();

        submit.addActionListener(e -> {

            String status = p.isSelected() ? "Present" : "Absent";

            att.add(name.getText(), subject.getText(), date.getText(), status);

            JOptionPane.showMessageDialog(f, "Stored");
        });

        f.setVisible(true);
    }
}
```

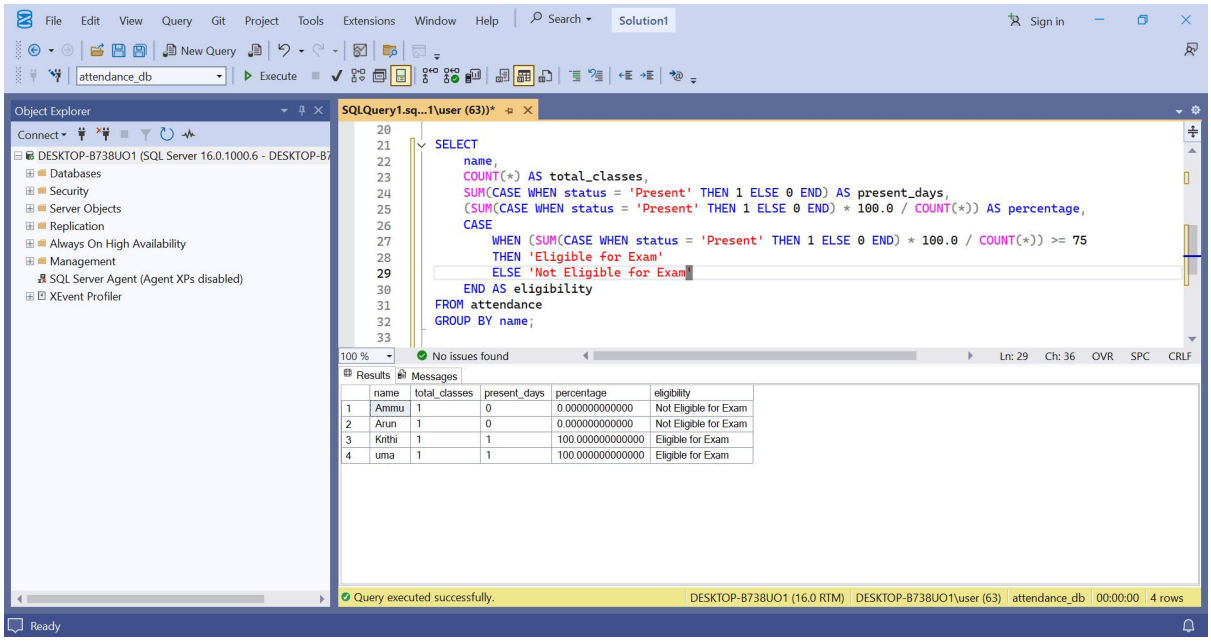
E. Main Application

```
public class MainApp {

    public static void main(String[] args) {

        Attendance att = new Attendance();

        att.markAttendance("Krithi", "OOSE", "22-03-2026", "Present");

        System.out.println(att.viewAttendance("Krithi"));
    }
}
```

OUTPUT:







RESULT:

Thus, the Student Attendance Management System was successfully implemented as per the detailed design.

EXP NO: 09	IMPROVE THE REUSABILITY AND MAINTAINABILITY OF THE SOFTWARE SYSTEM BY APPLYING APPROPRIATE DESIGN PATTERNS

PROJECT TITLE: STUDENT ATTENDANCE MANAGEMENT SYSTEM

DESIGN PATTERNS USED

- Singleton Pattern – Database Connection
- Factory Pattern – User Creation
- Observer Pattern – Notification System

1. SINGLETON PATTERN (DBCONNECTION)

Ensures only one instance of database connection is created.

```
public class DBConnection {

   private static DBConnection instance;

   private DBConnection() {
      System.out.println("Connection Created");
   }

   public static DBConnection getInstance() {
      if (instance == null) {
         instance = new DBConnection();
      }
      return instance;
   }
}
```

2. FACTORY PATTERN (USERFACTORY)

Used to create different types of users dynamically.

```
public class UserFactory {

   public static User getUser(String type, String name) {

      if (type.equalsIgnoreCase("student"))
         return new Student(name);

      else if (type.equalsIgnoreCase("staff"))
```

```
            return new Staff(name);

        else if (type.equalsIgnoreCase("admin"))
            return new Admin(name);

        return null;
    }
}
```

3. OBSERVER PATTERN (ATTENDANCE NOTIFICATION)

Used to notify users when attendance is marked.

Observer Interface

```
public interface Observer {
    void update(String message);
}
```

Student Observer

```
public class StudentObserver implements Observer {

    String name;

    public StudentObserver(String name) {
        this.name = name;
    }

    public void update(String message) {
        System.out.println("Notification to " + name + ": " + message);
    }
}
```

Attendance (Subject)

```
import java.util.ArrayList;
import java.util.List;

public class Attendance {

    List<Observer> observers = new ArrayList<>();

    public void addObserver(Observer o) {
        observers.add(o);
    }

    public void markAttendance() {
        notifyObservers("Attendance Marked");
    }

    public void notifyObservers(String msg) {
```

```
        for (Observer o : observers) {
            o.update(msg);
        }
    }
}
```

4. User Classes

```
public abstract class User {
    String name;
    public abstract void role();
}
public class Student extends User {

    public Student(String name) {
        this.name = name;
    }

    public void role() {
        System.out.println(name + " is Student");
    }
}
public class Staff extends User {

    public Staff(String name) {
        this.name = name;
    }

    public void role() {
        System.out.println(name + " is Staff");
    }
}
public class Admin extends User {

    public Admin(String name) {
        this.name = name;
    }

    public void role() {
        System.out.println(name + " is Admin");
    }
}
```

5. Main Pattern Demo

```
public class MainPatternDemo {

    public static void main(String[] args) {

        DBConnection db1 = DBConnection.getInstance();
        DBConnection db2 = DBConnection.getInstance();
```

```
        if (db1 == db2) {
            System.out.println("Singleton working");
        }

        User u1 = UserFactory.getUser("student", "Krithi");
        User u2 = UserFactory.getUser("staff", "Anu");

        u1.role();
        u2.role();

        Attendance a = new Attendance();

        StudentObserver s1 = new StudentObserver("Krithi");
        StudentObserver s2 = new StudentObserver("Anu");

        a.addObserver(s1);
        a.addObserver(s2);

        a.markAttendance();
    }
}
```

OUTPUT:

Connection Created

Singleton working

Krithi is Student

Anu is Staff

Notification to Krithi: Attendance Marked

Notification to Anu: Attendance Marked

RESULT:

Thus, appropriate design patterns were successfully applied to improve the reusability and maintainability of the system.

EXP NO: 10	IMPLEMENT THE MODIFIED SYSTEM AND TEST IT FOR VARIOUS SCENARIOS

PROJECT TITLE: **STUDENT ATTENDANCE MANAGEMENT SYSTEM**

STEP 1: Integrate Design Patterns

Design patterns are incorporated into the Student Attendance Management System to improve reusability, maintainability, and scalability of the application.

IMPLEMENTED FEATURES:

• Database Connection (Singleton Pattern)

• User Creation – Admin, Staff, Student (Factory Pattern)

• Attendance Notification System (Observer Pattern)

STEP 2: Regression Testing

Regression testing ensures that newly added features do not affect existing functionalities of the system.

TEST PLAN

Test Case ID	Test Scenario	Expected Result	Status
TC-01	Database Connection Singleton Test	Only one connection instance should exist	Pass
TC-02	User Creation via Factory Pattern	Different user types should be created correctly	Pass
TC-03	Attendance Notification	All users should receive notification when attendance is marked	Pass
TC-04	System Performance Test	System should handle multiple attendance entries efficiently	Pass
TC-05	Regression Testing (Existing Features)	Existing features should work without errors	Pass

TESTING TOOLS USED:

- JUnit (Unit Testing)
- Manual Testing (GUI and Database validation)
- Java (Development & Debugging)

STEP 3: Generate Final Report

Summary of Modifications:

- **Design Patterns Applied:** Improved reusability and maintainability
- **Testing Performed:** Unit Testing, Integration Testing, Regression Testing
- **Final Outcome:** The Student Attendance Management System is stable, efficient, and easy to maintain

RESULT:

Thus, the modified Student Attendance Management System was successfully implemented and tested across various scenarios.